Basic (single-agent support bot design) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `single_agent_design.md`: a one-sentence request is where most real AI features start, and the easiest mistake is building too much. Writing this design on paper first — assumptions, one agent, a handoff — trains sizing a design to the request. **If not:** your first real design conversation would be the first time you had to say what you're *not* building, and why.

Write your own design first, in `practice/single_agent_design.md`.

The request: "answer questions about our product docs, hand off to a human if unsure."

Solution

Story: the same shape, hardened exactly where a reviewer will push: what "unsure" really means, what the human receives, and what happens when the search tool itself fails. **If not:** the bot would confidently answer from loosely related docs, and go silent the day search is down.

Same shape, hardened at the two places a senior reviewer will actually push on:

- **Grounding-based "unsure" instead of self-reported confidence:** after retrieval, the prompt constrains the agent to answer only from the retrieved text, and to explicitly say "I don't know" if the retrieved text doesn't cover the question — not rely on the model's own stated confidence, which isn't reliably calibrated.
- **Handoff payload includes what was already tried:** the human doesn't start from zero — they see the question, what was searched, and *why* it triggered escalation. A score-based miss and a grounding-based miss mean different things: a score miss means the docs might not cover this topic at all; a grounding miss on a borderline retrieval might just mean the query needs rephrasing.
- **Explicit failure handling for the tool itself:** `search_docs` timing out or erroring is its own path (escalate with reason "docs search unavailable"), not silently swallowed or retried forever.
- **Escalation logging as a product signal:** every handoff is logged with its reason — this becomes a way to find gaps in the docs themselves (repeated escalations on the same topic means missing documentation, not just a bot problem).
- **Cost/latency named explicitly, and deliberately not solved yet:** one retrieval plus one generation call per question is cheap at the stated moderate volume. A caching layer is named

as a future addition *if* volume grows, not built now — this is "requirements → architecture is matching, not inspiration" from Core Concepts, applied to what *not* to build yet.

Why this is the stronger design, and what the weaker one gets wrong: The Basic version isn't wrong for the volume it assumes, but it leaves the actual "unsure" logic underspecified in a way that would fail in front of a reviewer — "if the score is low, hand off" doesn't address the case where retrieval finds *something* plausible-looking that isn't actually a real answer, which is precisely how a support bot ends up confidently stating something false. It also has no answer for "what happens when your search tool itself goes down," which is exactly the kind of failure-mode question Doc14 trains you to ask before anyone else does. The stronger version isn't more complicated for its own sake — every addition (the grounding check, the tool-failure path, escalation logging) traces back to a specific weakness the Basic version has, not to "what real production systems have."

Which one should you actually present in an interview? Start with the Basic version's shape — stated assumptions, one clean agent, no over-building — since that shows you can size a design to a one-sentence request. Then, before the interviewer even asks, volunteer the grounding-based unsure check and the tool-failure path. Naming those unprompted is what separates "can build a single-agent bot" from "can be trusted to own this in production," and it's exactly what this document's "Expected Behavior" section is asking you to do without being prompted.